K250512 SUMAIYA ABUBAKAR
10/21/26

DSA LAB 06 TASKS

QUES 01:

```
#include <iostream>
#include <string>
using namespace std;

int N, targetSum;
string targetCode;
string code = "";

bool used[10] = {false};
bool found = false;
bool anyValid = false;

void generate(int position, int sum);
void tryDigit(int digit, int position, int sum);

void tryDigit(int digit, int position, int sum)
{

  if (digit > 9)
  {
    return;
  }

  if (!(position == 0 && digit == 0) && !used[digit])
  {
    used[digit] = true;
    code = code + char('0' + digit);

    generate(position + 1, sum + digit);

    code.pop_back();
    used[digit] = false;
  }

  tryDigit(digit + 1, position, sum);
}

void generate(int position, int sum)
{
```

DSA LAB 06 TASKS

```
    if (position == N)
    {
        if (sum == targetSum)
        {
            cout << code << endl;
            anyValid = true;

            if (code == targetCode)
            {
                found = true;
            }
        }

        return;
    }

    tryDigit(0, position, sum);
}

int main()
{
    cout << "Enter number of positions: ";
    cin >> N;

    if (N < 1 || N > 10)
    {
        cout << "N must be between 1 and 10." << endl;
        return 0;
    }

    cout << "Enter target sum: ";
    cin >> targetSum;

    cout << "Enter target code: ";
    cin >> targetCode;

    cout << "\nValid lock codes:\n";

    generate(0, 0);

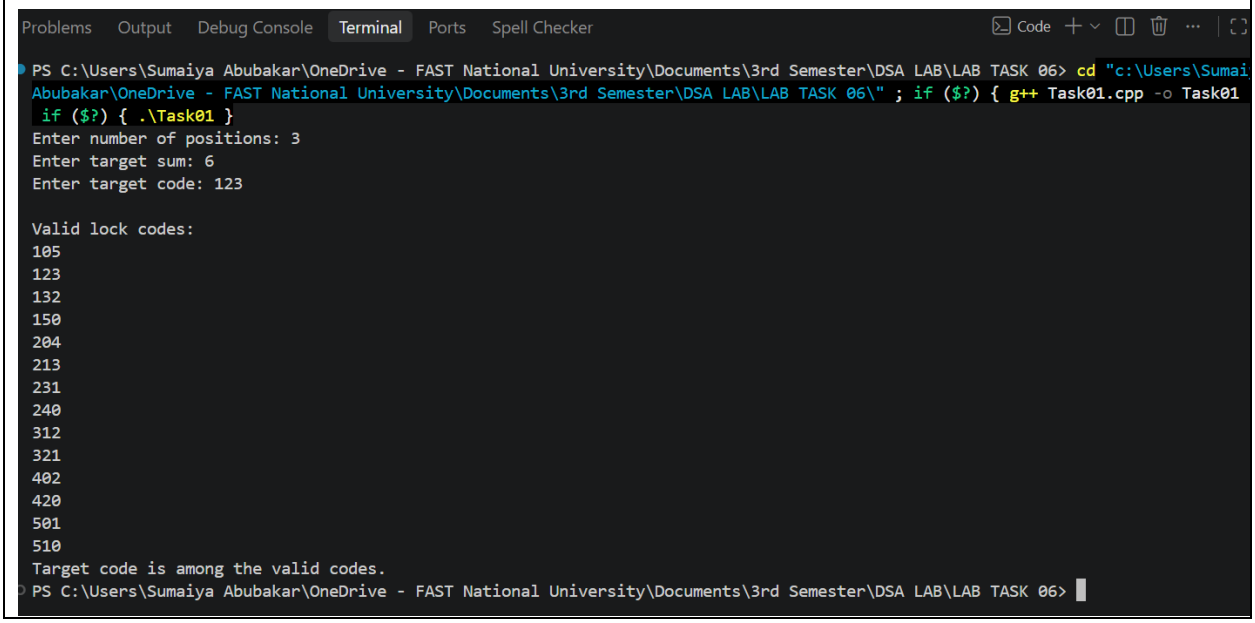
    if (!anyValid)
```

DSA LAB 06 TASKS

```
  {
    cout << "No valid codes found." << endl;
  }

  if (found)
  {
    cout << "Target code is among the valid codes." << endl;
  }
  else
  {
    cout << "Target code is NOT among the valid codes." << endl;
  }

  return 0;
}
```



QUES 02:

```
#include <iostream>
using namespace std;

struct Runner
{
  int stamina;
```

DSA LAB 06 TASKS

```
    int speed;
};

bool canFollow(Runner curr, Runner next)
{
    return curr.stamina + curr.speed >= next.stamina;
}

void generateOrders(Runner runners[], int n, int order[], bool used[], int pos, double avg)
{
    if (pos == n)
    {
        cout << "Running order: ";

        for (int i = 0; i < n; i++)
        {
            cout << "R" << order[i] + 1 << " ";
        }

        cout << endl;
        return;
    }

    for (int i = 0; i < n; i++)
    {
        if (used[i] == true)
        {
            continue;
        }

        if (pos > 0)
        {
            int prev = order[pos - 1];
            if (!canFollow(runners[prev], runners[i]))
            {
                continue;
            }

            if(canFollow(runners[i],runners[prev]))
            {
                continue;
            }
```

DSA LAB 06 TASKS

```
    }

    if (pos >= n - 2 && runners[i].stamina < avg)
    {
      continue;
    }
    order[pos] = i;
    used[i] = true;

    generateOrders(runners, n, order, used, pos + 1, avg);

    used[i] = false;
  }
}

int main()
{
  int n = 4;

  Runner runners[4] =
    {
      {10, 5},
      {13, 8},
      {7, 4},
      {15, 6}};

  int order[4];
  bool used[4] = {false};

  int totalStamina = 0;

  for (int i = 0; i < n; i++)
  {
    totalStamina += runners[i].stamina;
  }

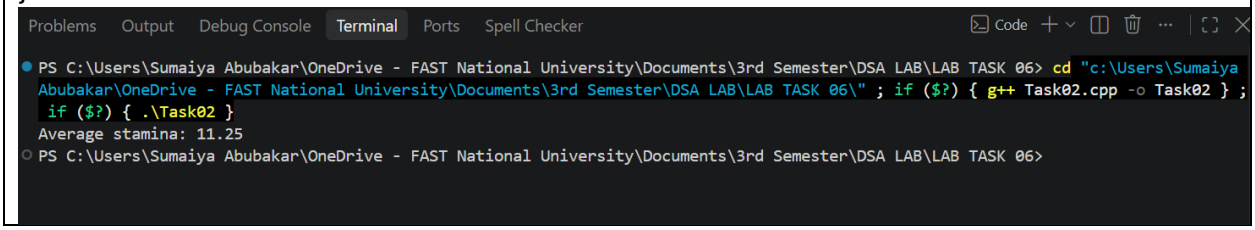
  double avg = (double)totalStamina / n;

  cout << "Average stamina: " << avg << endl;

  generateOrders(runners, n, order, used, 0, avg);
```

DSA LAB 06 TASKS

```
    return 0;
}
```



QUES 03:

```
#include <iostream>
#include <string>
using namespace std;

struct Student
{
    string name;
    string category;
    int priority;
    int minTokens;
    int maxTokens;
    int allocated;
};

bool checkStudentRestriction(Student students[]);
bool AllocateTokens(Student students[], int n, int index, int threshold, int totalTokens);
bool checkPriorityTotal(Student students[], int n, int threshold);
bool CheckMinMax(Student students[], int n);
bool CheckTokenSupply(Student students[], int n, int totalTokens);
void displayCategoryTotals(Student students[], int n);
bool checkCategoryLimit(Student students[], int n);

bool checkStudentRestriction(Student students[])
{
    return students[0].allocated > students[1].allocated;
}

bool AllocateTokens(Student students[], int n, int index, int threshold, int totalTokens)
{
    if (index == n)
    {
```

DSA LAB 06 TASKS

```
    return checkPriorityTotal(students, n, threshold) && CheckMinMax(students, n) &&
CheckTokenSupply(students, n, totalTokens)  && checkCategoryLimit(students, n) &&
checkStudentRestriction(students);
  }

  for (int t = students[index].minTokens; t <= students[index].maxTokens; t++)
  {
    students[index].allocated = t;

    bool valid = true;

    for (int j = 0; j < index; j++)
    {
      if (students[j].category == students[index].category && students[j].allocated == t)
      {
        valid = false;
        break;
      }

      if (students[j].priority == 1 && students[index].priority == 0 && students[j].allocated
< t)
      {
        valid = false;
        break;
      }

      if (students[j].priority == 0 && students[index].priority == 1 && students[j].allocated
> t)
      {
        valid = false;
        break;
      }
    }

    if (valid)
    {
      if (AllocateTokens(students, n, index + 1, threshold, totalTokens))
      {
        return true;
      }
    }
```

DSA LAB 06 TASKS

```
      students[index].allocated = 0;
   }

   return false;
}

bool checkPriorityTotal(Student students[], int n, int threshold)
{
   int hightotal = 0;
   int lowtotal = 0;

   for (int i = 0; i < n; i++)
   {
      if (students[i].priority == 1)
      {
         hightotal = hightotal + students[i].allocated;
      }
      else
      {
         lowtotal = lowtotal + students[i].allocated;
      }
   }

   return hightotal - lowtotal >= threshold;
}

bool CheckMinMax(Student students[], int n)
{
   bool minFound=false;
   bool maxFound=false;

   for(int i=0;i<n;i++)
   {
      if(students[i].allocated==students[i].minTokens)
      {
         minFound=true;
      }

      if(students[i].allocated==students[i].maxTokens)
      {
         maxFound=true;
      }
```

DSA LAB 06 TASKS

```
    }

    return maxFound && minFound;
}

bool CheckTokenSupply(Student students[], int n, int totalTokens)
{
    int totalAllocated=0;

    for(int i=0;i<n;i++)
    {
        totalAllocated=totalAllocated+students[i].allocated;
    }

    return totalAllocated<=totalTokens;
}

void displayCategoryTotals(Student students[], int n)
{
    int breakfast = 0;
    int lunch = 0;
    int dinner = 0;

    for (int i = 0; i < n; i++)
    {
        if (students[i].category == "Breakfast")
        {
            breakfast += students[i].allocated;
        }
        else if (students[i].category == "Lunch")
        {
            lunch += students[i].allocated;
        }
        else if (students[i].category == "Dinner")
        {
            dinner += students[i].allocated;
        }
    }

    cout << "Breakfast total: " << breakfast << endl;
    cout << "Lunch total: " << lunch << endl;
    cout << "Dinner total: " << dinner << endl;
```

DSA LAB 06 TASKS

```
}

bool checkCategoryLimit(Student students[], int n)
{
   int breakfast = 0;

   for (int i = 0; i < n; i++)
   {
      if (students[i].category == "Breakfast")
      {
         breakfast += students[i].allocated;
      }
   }

   return breakfast <= 5;
}

int main()
{
   const int n = 4;

   Student students[n] =
   {
      {"S1", "Breakfast", 1, 2, 5, 0},
      {"S2", "Breakfast", 0, 1, 4, 0},
      {"S3", "Lunch",     1, 2, 5, 0},
      {"S4", "Dinner",    0, 1, 3, 0}
   };

   int threshold = 2;
   int totalTokens = 10;

   // Find the maximum possible allocation
   int maxPossible = 0;

   for (int i = 0; i < n; i++)
   {
      maxPossible += students[i].maxTokens;
   }

   int maxEmergency = maxPossible - totalTokens;
```

DSA LAB 06 TASKS

```
if (maxEmergency < 0)
{
   maxEmergency = 0;
}

// Try emergency amounts from smallest to largest
bool found = false;
int emergencyUsed = 0;

for (int e = 0; e <= maxEmergency; e++)
{
   if (AllocateTokens(students, n, 0,
                threshold, totalTokens + e))
   {
      found = true;
      emergencyUsed = e;
      break;
   }
}

// Display the result
if (found)
{
   cout << "\nValid allocation found!" << endl;

   cout << "\nStudent Allocations:" << endl;

   int totalAllocated = 0;

   for (int i = 0; i < n; i++)
   {
      cout << students[i].name << " ("
         << students[i].category << "): "
         << students[i].allocated
         << " tokens" << endl;

      totalAllocated += students[i].allocated;
   }

   cout << "\nCategory Totals:" << endl;
   displayCategoryTotals(students, n);
```

DSA LAB 06 TASKS

```
    cout << "\nEmergency tokens used: "
         << emergencyUsed << endl;

    cout << "Unused tokens: "
         << totalTokens + emergencyUsed - totalAllocated
         << endl;
  }
  else
  {
    cout << "No valid allocation found!" << endl;
  }

  return 0;
}
```

```
Problems  Output  Debug Console  Terminal  Ports  Spell Checker                Code

PS C:\Users\Sumaiya Abubakar\OneDrive - FAST National University\Documents\3rd Semester\DSA LAB\LAB TASK 06> cd "c:\Users\Sumaiya
Abubakar\OneDrive - FAST National University\Documents\3rd Semester\DSA LAB\LAB TASK 06\" ; if ($?) { g++ Task03.cpp -o Task03 } ;
 if ($?) { .\Task03 }

Valid allocation found!

Student Allocations:
S1 (Breakfast): 2 tokens
S2 (Breakfast): 1 tokens
S3 (Lunch): 5 tokens
S4 (Dinner): 1 tokens

Category Totals:
Breakfast total: 3
Lunch total: 5
Dinner total: 1

Emergency tokens used: 0
Unused tokens: 1
PS C:\Users\Sumaiya Abubakar\OneDrive - FAST National University\Documents\3rd Semester\DSA LAB\LAB TASK 06>
```

QUES 04:

```
#include <iostream>
using namespace std;

const int rows = 4;
const int cols = 4;

int people[rows][cols] = {
  {8, 12, -1, 6},
  {5, 10, 7, -1},
  {-1, 9, 15, 4},
```

DSA LAB 06 TASKS

```
   {6, -1, 8, 11}};

int units[rows][cols] = {
   {2, 3, -1, 2},
   {1, 3, 2, -1},
   {-1, 2, 4, 1},
   {2, -1, 2, 3}};

bool highRisk[rows][cols] = {
   {1, 0, 0, 0},
   {0, 0, 1, 0},
   {0, 1, 0, 0},
   {0, 0, 0, 1}};

int capacity;
int unitsUsed = 0;
int totalPeople = 0;
int bestPeople = 0;
int bestunits = 0;
int highRiskCount = 0;
bool foundSolution = false;

bool selected[rows][cols] = {false};
bool bestSelected[rows][cols] = {false};

bool checkRows()
{
   for (int i = 0; i < rows; i++)
   {
      bool hasPeople = false;
      bool hasSelection = false;

      for (int j = 0; j < cols; j++)
      {
         if (people[i][j] != -1)
         {
            hasPeople = true;
         }

         if (selected[i][j] == true)
         {
            hasSelection = true;
```

DSA LAB 06 TASKS

```
        }
      }

      if (hasPeople && !hasSelection)
      {
        return false;
      }
    }

    return true;
}
void backtrack(int idx)
{
    if (idx == rows * cols)
    {
      if (checkRows() && (!foundSolution || totalPeople > bestPeople))
      {
        foundSolution = true;
        bestPeople = totalPeople;
        bestunits = unitsUsed;

        for (int i = 0; i < rows; i++)
        {
          for (int j = 0; j < cols; j++)
          {
            bestSelected[i][j] = selected[i][j];
          }
        }
      }
      return;
    }

    int row = idx / cols;
    int col = idx % cols;

    if (people[row][col] == -1)
    {
      backtrack(idx + 1);
      return;
    }

    int extraUnit = 0;
```

DSA LAB 06 TASKS

```
  if (highRisk[row][col] == 1 && highRiskCount == 1)
  {
    extraUnit = 1;
  }

  if (unitsUsed + units[row][col] + extraUnit <= capacity)
  {
    selected[row][col] = true;
    unitsUsed = unitsUsed + units[row][col] + extraUnit;
    totalPeople = totalPeople + people[row][col];

    if (highRisk[row][col] == 1)
    {
      highRiskCount++;
    }

    backtrack(idx + 1);

    selected[row][col] = false;
    unitsUsed = unitsUsed - units[row][col] - extraUnit;
    totalPeople = totalPeople - people[row][col];

    if (highRisk[row][col] == 1)
    {
      highRiskCount--;
    }
  }

  backtrack(idx + 1);
}
int main()
{
  cout << "\bPeople matrix: ";
  for (int i = 0; i < rows; i++)
  {
    for (int j = 0; j < cols; j++)
    {
      cout << people[i][j] << " ";
    }

    cout << endl;
```

DSA LAB 06 TASKS

```
}

cout << "\nRescue units matrix: ";
for (int i = 0; i < rows; i++)
{
   for (int j = 0; j < cols; j++)
   {
      cout << units[i][j] << " ";
   }

   cout << endl;
}

cout << "\nEnter rescue unit capacity: ";
cin >> capacity;

// ADDITION 2: Start backtracking from the first cell
backtrack(0);

if (!foundSolution)
{
   cout << "\nNo valid allocation exists." << endl;
}
else
{
   cout << "\nSelected areas (row, column): ";

   for (int i = 0; i < rows; i++)
   {
      for (int j = 0; j < cols; j++)
      {
         if (bestSelected[i][j] == true)
         {
            cout << "(" << i + 1 << ", " << j + 1 << ") ";
         }
      }
   }

   cout << "\nTotal people rescued: " << bestPeople;
   cout << "\nTotal rescue units used: " << bestunits;
   cout << "\nRemaining rescue units: " << capacity - bestunits;
}
```

DSA LAB 06 TASKS

```
    return 0;
}
```

